



Машинное обучение

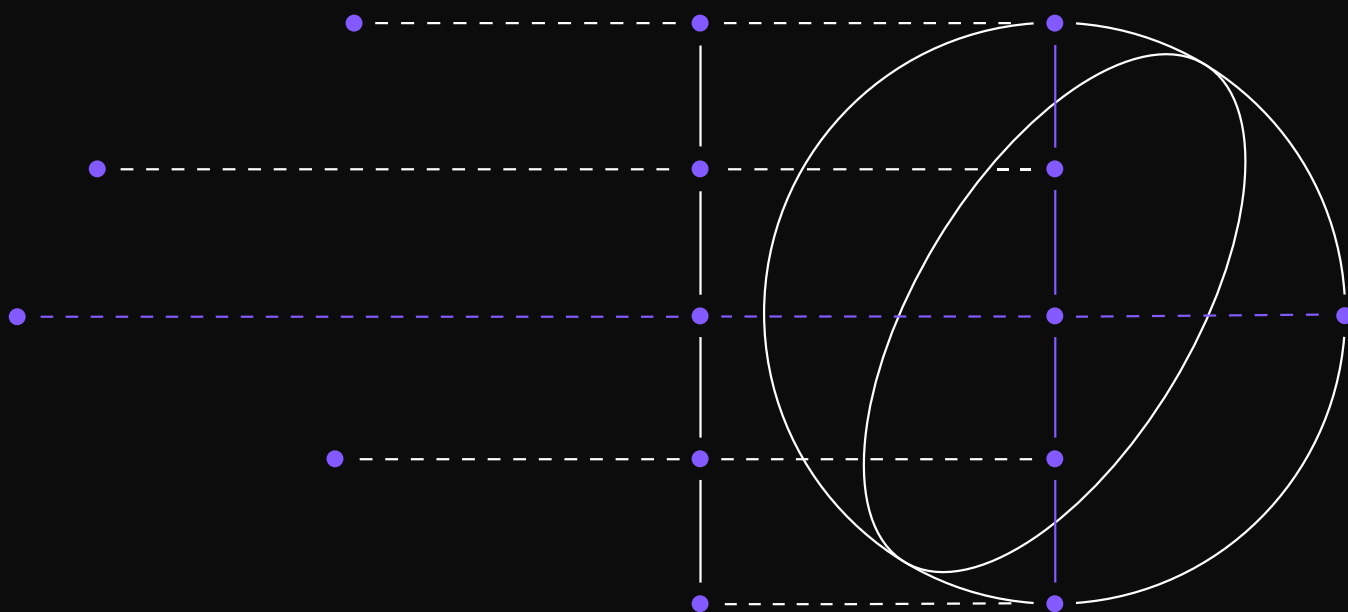
Лонгрид

Production ML

01

Инструменты разработки

ЭКСПЛУАТАЦИЯ И ВНЕДРЕНИЕ МОДЕЛЕЙ
МАШИННОГО ОБУЧЕНИЯ (PRODUCTION ML)



Контекст	Production ML начинается не с архитектуры нейросети, а с довольно скучных вещей вроде терминала, SSH, Git, структуры проекта, менеджера зависимостей и контейнера. Именно они превращают «мой ноутбук с экспериментом» в сервис, который работает у всех и везде одинаково. В этом случае инженер отличается от «человека, который умеет обучать модели» владением этими инструментами.
Применение	Из этого лонгрида ты узнаешь, как общаться с Linux через терминал и не бояться чёрного экрана, подключаться к удалённой машине по SSH и не терять процесс обучения модели при обрыве связи, а также как запустить и отладить сервис. А ещё как упаковать этот сервис в Docker и поднять весь стек одной командой через Compose.

Командная строка Linux

Случается, что ML-инженер полтора месяца обучает модель так, что ноутбук с экспериментами — произведение искусства. Но когда код переходит к коллеге, тот получает кучу ошибок из-за другой версии Python, забытой вручную установленной библиотеки и абсолютного пути **/Users/aleksey/Downloads/data_final(3)**. Это не выдуманная история, а типичный день из жизни ML-инженера, который не разобрался с инструментами.

Чтобы не допускать таких проблем, нужно понимать работу командой строки. Для этого начнём с её главной составляющей — оболочки.

ОПРЕДЕЛЕНИЕ

Shell (оболочка) — это интерфейс для общения с операционной системой через команды. Работает он так: ты пишешь текст, shell превращает его в системные вызовы и показывает результат.

Кажется, что графического интерфейса достаточно. Поэтому разберём, зачем нужна оболочка:

- на сервере графического интерфейса нет — есть только терминал по SSH;
- команды легко автоматизировать — то, что ты набрал(а) руками, можно сохранить в скрипт;
- терминал помогает управлять файлами и процессами быстрее, чем мышкой;
- всё, что происходит в CI/CD и внутри контейнера, — это тоже команды shell.

Самые распространённые оболочки:

- в Linux/macOS — Bash и Zsh;
- в Windows — PowerShell.

НАВИГАЦИЯ ПО ФАЙЛОВОЙ СИСТЕМЕ

Команда	Что делает
<code>pwd</code>	Выводит текущую рабочую директорию
<code>ls</code>	Выводит список элементов в директории
<code>cd</code>	Переходит в директорию
<code>tree</code>	Выводит дерево каталогов
<code>find</code>	Ищет файлы по заданному критерию

ПРИМЕР

```
> pwd
/root/week-1-development-tools

> ls -l
README.md  project_structure.ini  test

> cd test

> tree
.
├── file1.txt
├── nested1
│   ├── file2.yaml
│   └── file3.bin
└── nested2
    ├── file4.txt
    └── file5.py

> find . -type f -name "*.txt"
./nested2/file4.txt
./file1.txt
```

РАБОТА С ФАЙЛАМИ И ДИРЕКТОРИЯМИ

Команда	Что делает
<code>touch</code>	Создаёт пустой файл
<code>mkdir</code>	Создаёт новую директорию
<code>cp</code>	Копирует файл или директорию
<code>mv</code>	Перемещает или переименовывает
<code>rm</code>	Удаляет файл или директорию

ПРИМЕР

```
> touch file6.py

> mkdir nested3

> cp file6.py nested3

> cp -r nested2 nested3

> mv nested3/nested2 nested3/nested4

> rm -r file6.py nested3/nested4
```

Обрати внимание на флаг **-r** (recursive): без него **cp** и **rm** не тронут директории.

Инсайт



В Linux нет «Корзины»: **rm -rf** удаляет молча, сразу и навсегда. Поэтому, прежде чем нажать Enter на команде с **rm -rf**, перечитай путь ещё раз. Особенно в случае, если перед ним стоит переменная, которая может оказаться пустой.

ПРОСМОТР СОДЕРЖИМОГО ФАЙЛОВ

Команда	Что делает
<code>cat</code>	Выводит содержимое файла целиком
<code>head</code>	Выводит первые строки файла
<code>tail</code>	Выводит последние строки файла
<code>less</code>	Интерактивный просмотр содержимого
<code>wc</code>	Подсчёт строк, слов и символов
<code>vim / nano</code>	Консольный текстовый редактор

ПРИМЕР

```
> cat file1.txt
Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.

> head -n 2 file1.txt
Beautiful is better than ugly.
Explicit is better than implicit.

> tail -n 1 file1.txt
Complex is better than complicated.

> wc file1.txt
 4  20 132 file1.txt
```



Для просмотра содержимого также полезна команда, которая показывает новые строки лога по мере их появления: `tail -f logfile.log`. Обычно её используют при отладке сервиса, когда нужно понимать, что происходит в режиме реального времени.

КОМБИНИРОВАНИЕ КОМАНД, ПАЙПЫ И ПЕРЕНАПРАВЛЕНИЯ

Shell — это полноценный язык, в котором есть функции, циклы и условия. И хотя отдельные команды маленькие и делают одну вещь, их можно соединять в цепочки. Здесь и раскрывается настоящая сила shell.

Команда	Что делает
<code>;</code>	Последовательно объединяет команды
<code>&&</code>	Выполняет команду в случае, только если предыдущая успешна
<code> </code>	Выполняет команду в случае, только если предыдущая упала
<code> </code>	Направляет вывод одной команды на вход другой через pipe
<code><, > и 2></code>	Перенаправляет потоки stdin , stdout и stderr в файл
<code>VAR=value</code>	Задаёт переменные. Читается как \$VAR

ПРИМЕР

```
> pwd; ls
/root/week-1-development-tools
README.md  project_structure.ini  test

> mkdir test2 && cd test2

> cd test3 || echo "No such directory"
No such directory

> head -n 2 file1.txt | wc
      2      10      65

> cat file1.txt file1.txt > file7.txt

> A=32
> B=10
> echo $(( $A+$B ))
42

> test $A -gt $B && echo $A || echo $B
32
```



Философия Unix в том, что каждая программа делает одну вещь, но делает её хорошо. А общаются программы через текстовые потоки. Поэтому при решении какой-либо задачи проще написать пять простых команд, объединённых в конвейер, чем пытаться найти одну команду, которая полностью решит задачу.

ОБРАБОТКА ТЕКСТА И СТРОК

Команда	Что делает
<code>echo</code>	Выводит аргумент или значение переменной
<code>printf</code>	Форматированный вывод текста
<code>grep</code>	Поиск и фильтрация строк по шаблону
<code>cut</code>	Извлечение полей или диапазонов символов
<code>awk</code>	Обработка и форматирование строк по шаблонам
<code>tr</code>	Замена или удаление символов
<code>sed</code>	Потоковый редактор для замены текста
<code>rev</code>	Разворот строк задом наперёд
<code>sort</code>	Сортировка строк
<code>uniq</code>	Удаление повторяющихся строк
<code>jq</code>	Фильтрация и обработка JSON

ПРИМЕР

```
> echo "Hello, World!"
Hello, World!

> printf "Name: %s, Age: %d\n" "Alice" 25
Name: Alice, Age: 25

> cat file1.txt | grep "lex"
Simple is better than complex.
Complex is better than complicated.

> cut -f5 -d " " file1.txt | head -n 2
ugly.
implicit.

> echo "hello world" | tr 'a-z' 'A-Z'
HELLO WORLD

> sed 's/better/worse/' file1.txt | head -n 1
Beautiful is worse than ugly.

> echo -e "B\nA\nA\nC\nB" | sort | uniq -c
      2 A
      2 B
      1 C

> echo '{"name":"Alice","age":30,"city":"Paris"}' | jq
'.name'
"Alice"
```

→ Ответы сервиса и логи — это, как правило, JSON. Чтобы разобрать их прямо в терминале, при отладке сервиса удобнее использовать команду **jq**.

РАБОТА С ПРОЦЕССАМИ И СИСТЕМОЙ

Команда	Что делает
ps	Список активных процессов
top / htop	Интерактивно мониторит процессы и ресурсы
kill / pkill	Отправляет сигнал процессу
df	Показывает свободное и занятое место на дисках
du	Показывает размер файлов и директорий
free	Показывает использование оперативной памяти и swap
nvidia-smi	Помогает просматривать и управлять состоянием GPU, их памятью, загрузкой и процессами

ПРИМЕР

```
> ps -aux

> top

> kill -9 <PID>
> pkill -f python

> df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/vda1        500G   23G   478G   5% /

> du -h
4.0K    ./nested1

> free -h
              total        used        free
shared buff/cache available
Mem:          54Gi         3.2Gi         32Gi         127Mi
19Gi          50Gi
Swap:          0B           0B           0B
```

Эти команды помогут получить набор приборов для понимания состояния системы. Типичный паттерн при запуске серьёзной задачи обучения модели: дополнительно открыть терминалы с `htop` и `nvidia-smi` для мониторинга ресурсов.

УПРАВЛЕНИЕ ПРАВАМИ И ПОЛЬЗОВАТЕЛЯМИ

Команда	Что делает
<code>chown</code>	Помогает изменить владельца файла или директории
<code>chmod</code>	Помогает изменить права доступа
<code>whoami</code>	Выводит текущего пользователя
<code>id</code>	Выводит информацию о пользователе и группах

ПРИМЕР

```
> whoami
root

> ls -lah file1.txt
-rw-r--r-- 1 root root 132 Aug 18 00:59 file1.txt

> chown alice file1.txt
> chmod +x file1.txt

> ls -lah file1.txt
-rwxr-xr-x 1 alice root 132 Aug 18 00:59 file1.txt
```



Строка `-rwxr-xr-x` читается по группам: индикатор директория или файл (**d** — это директория, **-** — это обычный файл, **l** — это ссылка).

Дальше три символа — права владельца, группы и всех остальных:

- где **r** — чтение;
- **w** — запись;
- **x** — исполнение.

ДРУГИЕ ПОЛЕЗНЫЕ КОМАНДЫ

Команда	Что делает
<code>clear</code>	Очищает терминал
<code>man</code>	Выводит справочник по команде
<code>sudo</code>	Выполняет команды с правами суперпользователя
<code>date</code>	Выводит текущую дату и время
<code>tar, zip/unzip, gzip</code>	Создаёт, показывает и распаковывает архивы
<code>source</code>	Выполняет команды из файла в текущей оболочке

ПРИМЕР

```
> man ls

> sudo vim ~/.ssh/authorized_keys

> tar -czf archive.tar.gz file* dir*
> tar -xf archive.tar.gz

> source ~/.bashrc
```

Инсайт



Команда **man** — это первое место, куда стоит смотреть для изучения нужной команды. В этом справочнике есть все флаги, а не только те три, которые попали в верхний ответ на Stack Overflow. Если же `man`-страница кажется нечитаемой, стоит попробовать команду **tldr**.

SSH и удалённая разработка

ЗАЧЕМ РАБОТАТЬ НА УДАЛЁННОЙ МАШИНЕ

ОПРЕДЕЛЕНИЯ

Виртуальная машина (ВМ) — компьютерная система, эмулирующая возможности каких-либо вычислительных комплексов.

Удалённая разработка — работа с кодом и сервисами на машине, которая физически находится не у тебя, а в дата-центре или облаке.

Разберём четыре причины, по которым почти вся работа в Production ML происходит на удалённой машине.

- Доступ к мощным ресурсам — к серверным CPU, GPU и увеличенному объёму памяти, т. е. ко всему, к чему у ноутбука нет доступа.
- Единая среда разработки — окружение виртуальной машины совпадает с production, что снижает риск несовместимостей.
- Непрерывность работы — ВМ доступна в любое время и не зависит от того, закрыл ли ты крышку ноутбука.
- Безопасность и изоляция — код, данные и ключи хранятся на удалённом сервере; локальная машина используется только для доступа.

ЧТО ТАКОЕ SSH

ОПРЕДЕЛЕНИЕ

SSH (Secure Shell) — протокол для безопасного доступа к удалённым системам. SSH работает поверх TCP и по умолчанию использует порт 22.

SSH помогает:

- шифровать трафик,
- проходить аутентификацию по паролю или ключу,
- проводить туннелирование и проброс портов.

АУТЕНТИФИКАЦИЯ: ПАРОЛЬ ИЛИ КЛЮЧ

	По паролю	По ключам
Плюсы	Простота: не требует дополнительной настройки	• Устойчива к брутфорсу (подбор ключа невозможен). Секретная часть вообще не передаётся по сети. • Подходит для автоматизации
Минусы	• Подвержена брутфорс-атакам при слабых паролях. • Требуется регулярного ввода пароля, неудобно для автоматизации	Требуется настройка: генерация ключей и добавление публичного ключа на сервер

ВЫВОД

В реальной работе используются ключи. Пароль — это то, что ты вводишь один раз, чтобы положить на сервер публичный ключ и больше никогда не вводить пароль.

ОСНОВНЫЕ КОМАНДЫ

Команда	Действие
<code>ssh user@hostname</code>	Подключение к серверу
<code>ssh -p <port> user@hostname</code>	Указание порта
<code>scp file.txt user@hostname:/path/</code> <code>scp user@hostname:/path/file.txt .</code>	Копирование файлов
<code>scp -r project/ user@hostname:/path/</code>	Копирование директорий
<code>ssh user@hostname <command></code>	Исполнение команды на сервере
<code>ssh-keygen -t rsa -b 4096 -C "comment"</code>	Генерация пары ключей
<code>ssh-copy-id user@host</code>	Отправка публичного ключа на сервер
<code>ssh -i ~/.ssh/id_rsa user@hostname</code>	Явное указание ключа
<code>ssh-add -l</code>	Просмотр имеющихся ключей
<code>ssh-add ~/.ssh/id_rsa</code>	Добавление ключа

Инсайт



Пара ключей — это два файла:

- `id_rsa` — приватный ключ, он никогда и никуда не уезжает с твоей машины;
- `id_rsa.pub` — публичный ключ, его можно спокойно раздавать.

На сервер отправляют **публичный ключ**. Если ты хоть раз запустил приватный ключ в Git, это значит, что теперь он доступен всем пользователям репозитория. В этом случае для сохранения доступа придётся генерировать новый приватный ключ.

PORT FORWARDING

SSH умеет перенаправлять сетевой трафик через защищённый канал, создавая туннели между клиентом и сервером. Для этого используют два вида проброса портов: локальный и удалённый.

Локальный проброс (local forwarding) направляет локальный порт на сервер.

```
ssh -L 8080:localhost:80 user@server
```

Любой запрос к `localhost:8080` на клиенте перенаправляется на `server:80`.

Пример использования: доступ к сервисам или базам данных, недоступным напрямую.

Удалённый проброс (remote forwarding) направляет порт сервера на локальный компьютер.

```
ssh -R 9090:localhost:3000 user@server
```

Любой доступ к `server:9090` перенаправляется на `localhost:3000` клиента.

Пример: подключение дебаггера к процессу, запущенному удалённо.

КОНФИГУРАЦИЯ SSH

Каждый раз писать `ssh -p 5000 -i ~/.ssh/id_rsa user@example.com` — это утомительно. Вместо этого в SSH можно хранить настройки подключения в файле `~/.ssh/config`.

```
Host myserver                                # Указание алиаса
  HostName example.com                      # Адрес сервера
  User user                                 # Имя пользователя
  Port 5000                                 # Порт SSH
  ForwardAgent yes                          # Проброс SSH-ключей
  ServerAliveInterval 60                   # Кеер-alive
  IdentityFile ${HOME}/.ssh/id_rsa         # Приватный ключ
  LocalForward 8080 localhost:9096         # Проброс локального порта
  RemoteForward 5502 localhost:5502       # Проброс удалённого порта
```

После создания файла достаточно написать `ssh myserver`. Тот же алиас подхватят `scp`, VS Code и все остальные инструменты, работающие поверх SSH.

ДОЛГОЖИВУЩИЕ ПРОЦЕССЫ НА СЕРВЕРЕ

Вот классические грабли: ты запустил(а) обучение на удалённой машине, залогинившись по SSH, закрыл(а) ноутбук — и обучение умерло. Так и должно быть: **если SSH-сессия закрывается или прерывается по любой причине, запущенные в ней команды останавливаются**. Чтобы избежать этого, можно использовать два решения.

Решение 1. Использовать `nohup`.

Он позволяет запускать команду так, чтобы она продолжала работать после выхода из сессии.

```
nohup python train.py > output.log 2>&1 &
```

Здесь:

- `&` — запуск в фоне;
- `> output.log` и `2>&1` — перенаправление stdout и stderr в файл.

Решение 2. Использовать `tmux` или `screen`.

Это терминальные мультиплексоры, другими словами, виртуальные терминалы. Они помогают иметь несколько сессий, вкладок и окон, которые не зависят от подключения.

```
tmux new -s session_name # Создать новую сессию
tmux attach -t session_name # Подключиться к существующей
tmux detach # Отсоединиться, процесс продолжает работать
tmux ls # Список сессий
```

Инсайт



Заведи привычку: первое, что ты делаешь после `ssh` на сервер, — это `tmux attach || tmux new`. Тогда любой обрыв Wi-Fi в метро не будет стоить тебе нескольких часов обучения.

Разница между `nohup` и `tmux` в том, что к `tmux`-сессии можно вернуться и посмотреть, что происходит, продолжить работу, как если бы терминал был у тебя локально; `nohup` же оставляет тебе только лог-файл.

Git и совместная разработка

ЗАЧЕМ НУЖНА СИСТЕМА КОНТРОЛЯ ВЕРСИЙ

Пока ты один и проект маленький, то может казаться, что достаточно копий папки.

Реальность выглядит иначе: `model_final`, `model_final_v2`, `model_final_v2_РАБОЧАЯ`.

И уже через неделю никто не помнит, чем они отличаются. Поэтому для поддержания порядка в проекте используют систему контроля версий — VCS, или Version Control System.

Что даёт использование VCS:

- история изменений — возможность вернуться к любой предыдущей версии кода;
- безопасность работы — эксперименты в ветках не влияют на основную версию;
- совместная разработка — несколько человек могут работать над проектом параллельно;
- прозрачность — видно, кто и какие изменения внёс, удобно для ревью;
- интеграция с автоматизацией — CI/CD, тестирование и деплой напрямую из репозитория.

ОСНОВНЫЕ СУЩНОСТИ GIT

ОПРЕДЕЛЕНИЕ

Репозиторий (repository) хранит весь код проекта и историю изменений. Может быть локальным (на твоей машине) и удалённым (GitHub/GitLab).

Remote — удалённый репозиторий, общий для совместной работы.

Коммит (commit) — снимок текущего состояния файлов. Имеет уникальный SHA-хеш и сообщение с описанием изменений.

Staging area (индекс) — промежуточное место, где файлы готовятся к коммиту. Помогает закоммитить только выбранные изменения.

Ветка (branch) — отдельная линия разработки. Позволяет экспериментировать и параллельно работать с кодом.

HEAD — указатель на текущий коммит/ветку, с которой ты работаешь.

Ключевая идея, которая многих сбивает: файл в Git проходит три состояния —

рабочая директория



staging area



репозиторий

`git add` при этом переносит из первого во второе, а `git commit` — из второго в третье.

ОСНОВНЫЕ КОМАНДЫ

Команда	Действие
<code>git init</code>	Инициализация нового репозитория
<code>git clone <url></code>	Клонирование существующего репозитория
<code>git status</code>	Просмотр текущего состояния
<code>git log</code>	Просмотр истории коммитов
<code>git add <file></code>	Добавление изменений в индекс
<code>git commit -m "message"</code>	Создание коммита
<code>git diff</code>	Просмотр изменений
<code>git checkout <branch></code>	Переключение ветки
<code>git branch <branch></code>	Создание новой ветки
<code>git branch -d <branch></code>	Удаление ветки
<code>git branch -m <old> <new></code>	Переименование ветки
<code>git checkout -b <branch></code>	Создание ветки с переключением
<code>git push</code>	Отправка изменений на удалённый репозиторий

<code>git fetch</code>	Получение изменений с удалённого репозитория
<code>git pull</code>	Получение и автоматическое слияние изменений
<code>git remote -v</code>	Просмотр удалённых репозитория
<code>git remote add <name> <url></code>	Добавление нового удалённого репозитория
<code>git merge <branch></code>	Слияние ветки в текущую
<code>git rebase <branch></code>	Ребейз текущей ветки на другую

 `git status` — самая недооценённая команда. Она показывает состояние и подсказывает, какую команду выполнить дальше. Если ты не понимаешь, что происходит с репозиторием, то начни с её вызова.

ТРИ СПОСОБА СЛИТЬ ВЕТКИ

Разберём три способа объединить изменения из одной ветки в другую: **fast-forward**, **merge** и **rebase**.

Fast-forward.

Если в целевой ветке не появилось новых коммитов, Git просто передвигает указатель вперёд, история остаётся линейной, коммита слияния не появляется.

```
git checkout target_branch
git merge --ff-only source_branch
```

Merge.

Если обе ветки разошлись, Git создаёт отдельный merge-коммит с двумя родителями. История сохраняется как есть, со всеми развилками.

```
git checkout target_branch
git merge source_branch
```

Rebase.

Коммиты ветки переносятся поверх целевой, как будто изначально работа начиналась от её последнего состояния. История получается линейной, но коммиты новые, с другими хешами.

```
git checkout feature_branch
git rebase master
```

Часто при работе в **feature_branch** делают ребейз на основную ветку: это помогает поддерживать её актуальность и минимизировать число конфликтов.

Инсайт



Золотое правило: не проводи ребейз ветки, если над ней уже кто-то работает. Ребейз переписывает историю, и коллега, у которого старая версия этих коммитов, получит увлекательный вечер по разбору конфликтов. А вот внутри своей feature-ветки «ребейзи» сколько угодно.

ПЛАТФОРМЫ ДЛЯ СОВМЕСТНОЙ РАЗРАБОТКИ

К платформам для совместной разработки относят GitHub, GitLab и Bitbucket. Все они добавляют то, чего нет в Git: pull/merge requests, code review, issues, CI/CD и права доступа.

СОГЛАШЕНИЯ ОБ ИМЕНОВАНИИ

ОПРЕДЕЛЕНИЕ

Conventional commits — один из вариантов соглашения о названии коммитов.

Формат

`<type>[optional scope]: <description>`

ПРИМЕР

```
feat(parser): add CSV support
refactor: remove outdated toggles
```

Названия веток должны быть короткими, читаемыми и отражать цель:

- **feature/login-page** — новая функциональность;
- **hotfix/crash-on-start** — срочное исправление;
- **release/video-service-21** — релизная ветка.

Зачем это нужно. Такие соглашения:

- поддерживают читаемость истории проекта;
- облегчают совместную работу;
- упрощают автоматизацию, например автоматическую генерацию changelog по типам коммитов.

PRE-COMMIT HOOKS

ОПРЕДЕЛЕНИЕ

Pre-commit hooks — это скрипты, которые выполняются до того, как изменения попадут в репозиторий, то есть локально на твоей машине.

С помощью pre-commit hooks:

- проверяют форматирование и стиль кода (линтеры, black, flake8, ruff);
- запускают юнит-тесты или другие проверки;
- проверяют соответствие сообщений коммитов соглашениям, например conventional commits;
- предотвращают добавление чувствительных данных — ключей, паролей.

Конфигурация лежит в `.pre-commit-config.yaml`.

```
repos:
-   repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v2.3.0
    hooks:
    -   id: check-yaml
    -   id: end-of-file-fixer
    -   id: trailing-whitespace
-   repo: https://github.com/psf/black
    rev: 22.10.0
    hooks:
    -   id: black
```

Инсайт



Чтобы избежать попадания чувствительных данных в публичный репозиторий, можно создать хук, который проверяет отсутствие ключей, паролей, токенов и т. д. в коммите.

Структура проекта

Структура проекта упорядочивает файлы и папки — её используют:

- чтобы быстро ориентироваться в коде;
- разделить код, тесты, данные и документацию;
- облегчить воспроизводимость и стандартизацию.

Кроме папок и файлов, к структуре проекта относят:

- организацию версий — semantic versioning (MAJOR.MINOR.PATCH) помогает отслеживать изменения и совместимость;
- конфигурацию — `pyproject.toml` и `requirements.txt` улучшают воспроизводимость проекта;
- документацию — `README` и `docs/` объясняют структуру, назначение компонентов и правила работы;
- кодовые стандарты — единый стиль, линтеры, форматирование, соглашения по именованию;
- автоматизацию и тесты — тесты, CI/CD и `pre-commit hooks` поддерживают качество и стабильность проекта.

ПРАКТИЧЕСКИЕ ПРАВИЛА ВЕДЕНИЯ ПРОЕКТА

Эти правила помогут держать код, документацию и настройки в порядке.

- Используй единообразные имена директорий и файлов.
- Разделяй исходный код, тесты, документацию и вспомогательные файлы.
- Добавляй `README` с описанием цели проекта, зависимостей и инструкций по запуску.
- Храни конфигурацию окружения в проекте, например в таких файлах, как `requirements.txt`, `pyproject.toml`, `Dockerfile` и т. д.
- Используй `.gitignore` для исключения временных файлов, данных, кешей и локальных настроек.
- Организуй тесты так, чтобы их можно было запускать одной командой.
- Документируй структуру и принципы работы проекта в `docs/` или `wiki`.
- Следи за чистотой корневой директории по принципу минимум файлов вне каталогов.

Типичный скелет сервиса выглядит так.

```
my_service/
├── my_service/           # Исходный код пакета
│   ├── __init__.py
│   ├── api/
│   └── models/
├── config/              # Конфигурации
├── tests/               # Тесты
├── docs/                # Документация
├── .gitignore
├── .pre-commit-config.yaml
├── Dockerfile
├── docker-compose.yaml
├── pyproject.toml
└── README.md
```



Хороший README отвечает на три вопроса за 30 секунд: что это, как это запустить и как это проверить. Если новый человек не может поднять проект по README без личных консультаций, то значит, README не работает, каким бы длинным он ни был.

Работа в VS Code

VS Code — основная IDE этого курса. Её выбрали, потому что она:

- лёгкая и быстрая IDE с широкими возможностями настройки;
- поддерживает множества языков и расширений — Python, Docker и других;
- легко интегрируется с Git и системами контроля версий;
- удобно подключается к удалённым машинам через Remote SSH;
- объединяет поддержку отладчика, терминала и управления виртуальными окружениями в одном окне.

REMOTE-SSH

ОПРЕДЕЛЕНИЕ

Remote-SSH — расширение для VS Code, которое помогает подключаться к удалённому серверу через SSH и работать с файлами так, как если бы они находились на локальной машине. Если ты уже настроил(а) `~/.ssh/config`, то VS Code подхватит оттуда все алиасы.

Организация Python-проектов

УПРАВЛЕНИЕ ЗАВИСИМОСТЯМИ: PIP

ОПРЕДЕЛЕНИЕ

`pip` — это стандартный менеджер пакетов для Python. Он работает с PyPI (Python Package Index), устанавливает, обновляет и удаляет пакеты.

```
# Установка пакета
pip install requests

# Установка конкретной версии
pip install requests==2.31.0

# Обновление пакета
pip install --upgrade requests

# Установка из requirements.txt
pip install -r requirements.txt

# Просмотр установленных пакетов
pip list

# Сохранение зависимостей
pip freeze > requirements.txt

# Удаление пакета
pip uninstall requests
```

Зависимости перечисляются в текстовом файле `requirements.txt`.

```
requests==2.31.0
numpy>=1.25.0
jupyter
```

Недостатки `pip` как менеджера пакетов:

- нет автоматического разрешения конфликтов зависимостей;
- нет поддержки `lock`-файлов → сложно в точности повторить окружение;
- нет декларативного управления проектом (нет единого манифеста).

Инсайт



Команда `pip freeze > requirements.txt` — это ловушка, в которую попадают почти все. Она сохраняет в файл всё, что стоит в окружении, включая транзитивные зависимости и случайный `jupyter`, поставленный однажды «посмотреть». В итоге через месяц никто не может ответить, какие из 120 строк действительно нужны проекту.

ВИРТУАЛЬНЫЕ ОКРУЖЕНИЯ

ОПРЕДЕЛЕНИЕ

Виртуальное окружение — это изолированная среда для Python-проекта, в котором есть своя версия интерпретатора и свой набор библиотек.

Плюсы виртуального окружения:

- изолирует зависимости — у каждого проекта собственный набор библиотек;
- предотвращает конфликты версий — в нём разные версии пакетов могут сосуществовать;
- воспроизводит проект — легко запустить проект на другой машине;
- поддерживает стабильность локальной среды — защитить системный Python от неожиданных изменений.

Минусы виртуального окружения:

- не гарантирует production-среду — окружение ≠ контейнер (это решает Docker);
- не изолирует системные библиотеки — возможны проблемы при нативных зависимостях.

VENV

Это базовый инструмент, который встроен в Python и используется как основа у более продвинутых менеджеров пакетов.

```
#Создать окружение
python3 -m venv .venv

# Активировать окружение
source .venv/bin/activate

# Установить зависимости
pip install -r requirements.txt

# Сохранение зависимостей
pip freeze > requirements.txt

# Деактивировать окружение
deactivate
```

Внутри `.venv` нет никакой магии — это копия Python-интерпретатора, каталоги `include/` `lib` с пакетами (`site-packages`) и скрипты для активации. Активация всего лишь подменяет переменную `PYTHON`.

СБОРКА И ПУБЛИКАЦИЯ ПАКЕТОВ

Сборку пакетов используют:

- чтобы упаковать проект в переносимый формат (wheel/sdist);
- сделать пакет доступным для установки через pip/PyPI.

Базовые инструменты — `setup.py` + `setuptools/wheel`, а также `twine` для публикации.

```
#Создать wheel
python setup.py sdist bdist_wheel

# Загрузить на PyPI
twine upload dist/*
```

Проблемы сборки пакетов:

- ручная настройка — метаданные и зависимости прописываются руками;
- разделённые шаги — сборка и публикация выполняются отдельно;
- отсутствие стандарта — разные проекты имеют разную структуру setup.py.

PYPROJECT.TOML

ОПРЕДЕЛЕНИЕ

pyproject.toml — это современный стандарт конфигурации Python-проектов (PEP 518 / PEP 621). Единый файл, содержащий метаданные проекта, зависимости, настройки сборки и не только.

pyproject.toml решает проблемы:

- разделения **setup.py** / **requirements.txt** / **MANIFEST.in** по разным файлам;
- сложной сборки и публикации;
- отсутствия единообразного формата для инструментов.

```
[project]
name = "my-service"
version = "0.1.0"
description = "Demo Python service"
authors = [
    { name="Ivan Ivanov", email="ivan@example.com" }
]
license = { text = "MIT" }
readme = "README.md"
requires-python = ">=3.10"
dependencies = [
    "fastapi>=0.110.0,<0.111.0",
    "uvicorn[standard]>=0.22.0,<0.23.0",
    "requests>=2.31.0"
]

[tool.ruff]
line-length = 88
select = ["E", "F", "W", "C90"]
exclude = ["tests/fixtures", "build"]

[build-system]
requires = ["setuptools>=42", "wheel"]
build-backend = "setuptools.build_meta"
```

Обрати внимание на секцию `[tool.ruff]`: настройки линтера живут в том же файле. Так же туда переезжают `pytest`, `mypy`, `black` и остальные инструменты вместо россыпи `.flake8`, `setup.cfg` и `pytest.ini`.

СОВРЕМЕННЫЕ МЕНЕДЖЕРЫ ПАКЕТОВ: POETRY И UV

Менеджеры пакетов Poetry и uv — это инструменты «всё в одном» для Python-проектов. Они используют `pyproject.toml` как единый источник конфигурации и берут на себя:

- управление виртуальными окружениями,
- установку и обновление зависимостей с разрешением конфликтов,
- lock-файлы для воспроизводимости окружений,
- сборку wheel/sdist и публикацию на PyPI.

uv — это менеджер пакетов, с которым ты будешь работать на курсе. Его преимущества:

- включает «всё в одном» — окружение, зависимости, сборку, публикацию, запуск сервисов и задач;
- использует `pyproject.toml` как единый источник конфигурации проекта;
- имеет lock-файл — гарантирует воспроизводимость окружения;
- управляет dev-зависимостями — легко добавлять и обновлять dev-пакеты;
- имеет высокую производительность: написан на Rust и работает очень быстро;
- активно развивает и поддерживает сообщество.

```
# Создать новый проект
uv init

# Установить новый пакет
uv add requests

# Удалить пакет
uv remove requests

# Обновить окружение проекта
uv sync

# Запустить скрипт/сервис
uv run my_service.main:app

# Вывести дерево зависимостей
uv tree

# Собрать пакет
uv build

# Опубликовать на PyPI
uv publish

# Обновить lock-файл
uv lock
```



Главное отличие uv/Poetry от pip — это **lock-файл**. В **pyproject.toml** ты пишешь, что тебе нужен **fastapi>=0.110**, а в lock-файле фиксируются точные версии всех пакетов, включая транзитивные, с хешами. Именно lock-файл — ответ на вопрос «Почему у меня работает, а на сервере нет?». Поэтому обязательно коммить его в Git.

Запуск и отладка Python-сервиса

А теперь попрактикуемся на тестовом приложении [в репозитории](#). Для этого запустим сервис и отправим к нему первые тестовые запросы. Детали устройства проекта можно найти в README.md в репозитории.

Клонируем репозиторий и настроим окружение.

```
git clone git@git.culab.ru:courses/production-ml/materials-production-ml.git
cd materials-production-ml/lesson_1
uv sync
cp .env.example .env
```

Когда окружение готово, запускаем сервис. Для FastAPI-приложения это делает ASGI-сервер **uvicorn**.

```
uv run --env-file .env uvicorn service.main:app --host 0.0.0.0 --port 8000 --reload
```

Разберём аргументы:

- **--env-file .env** перед запуском загружает переменные окружения из файла .env;
- **service.main:app** — путь к объекту приложения: модуль, затем через двоеточие имя переменной;
- **--host 0.0.0.0** — слушать на всех интерфейсах, а не только на localhost (критично внутри контейнера, иначе снаружи сервис будет недоступен);
- **--port 8000** — порт;
- **--reload** — перезапуск при изменении файлов. Удобно в разработке, никогда в production.

Проверить, что сервис жив, проще всего через **curl**.

```
curl -s localhost:8000/health
# {"status":"ok"}

curl -s "localhost:8000/get?key=foo"
# 503 {"detail":"database is not available"}

curl -s -X POST localhost:8000/set \
  -H "Content-Type: application/json" \
  -d '{"key": "foo", "value": "bar"}'
# 503 {"detail":"database is not available"}
```

ОТЛАДКА

Печать через `print()` работает, но плохо масштабируется. Поэтому вместо него стоит использовать два других инструмента — отладчик VS Code и логирование.

Отладчик VS Code. Для работы с отладчиком нужно:

- 1) установить точку установки — кликнуть слева от номера строки;
- 2) запустить конфигурацию из файла `.vscode/launch.json`;
- 3) получить доступ к пошаговому выполнению кода, стеку вызовов и возможности выполнять произвольный Python-код прямо в контексте остановленного кода.

Подробнее о том, как настроить `.vscode/launch.json`, — [в официальной документации](#).

Логирование. В отличие от `print`, логгер даёт уровни (`DEBUG/INFO/WARNING/ERROR`), метки времени, имя модуля и настраиваемый вывод. Всё это конфигурируется из файла без правки кода. Именно поэтому в примере ниже сервис запускается с `--log-config ./config/logging_config.yaml`.

Инсайт



Возможность подключаться к удалённому серверу и отлаживать код прямо в VS Code через связку «Remote-SSH + отладчик» — это главная причина, по которой стоит потратить полчаса на настройку VS Code.

Для этого достаточно поставить breakpoint в файле, который физически лежит на сервере и отладить процесс, который работает на серверных GPU — интерфейс при этом ничем не отличается от локальной отладки. Если же контейнер запущен с проброшенным портом дебаггера, точно так же можно зайти отладчиком прямо внутрь контейнера.

Контейнеризация с Docker

ПРОБЛЕМА, КОТОРУЮ РЕШАЕТ DOCKER

В современной разработке важно обеспечить портативность, масштабируемость и изоляцию приложений. Здесь виртуальное окружение изолирует лишь Python-пакеты, но не трогает системные библиотеки, версию самого Python, переменные окружения и конфигурацию ОС. И как только проекту понадобится определённая версия CUDA или системного `libgomp`, то `venv` становится бесполезным. Поэтому в этом случае используют Docker.

ОПРЕДЕЛЕНИЯ

Контейнеризация — это технология, с помощью которой можно запускать приложения в изолированных средах, включающих все необходимые зависимости: библиотеки, системные утилиты, код и конфигурацию.

Docker — это платформа для контейнеризации, которая помогает упаковывать, распространять и запускать приложения в стандартизированных средах — **контейнерах**.

При традиционной виртуализации каждое приложение работает в отдельной виртуальной машине и со своей операционной системой. Контейнеры же работают иначе: они используют ядро хостовой операционной системы компьютера, на котором запущены. Поэтому они гораздо легче, быстрее и потребляют меньше ресурсов.

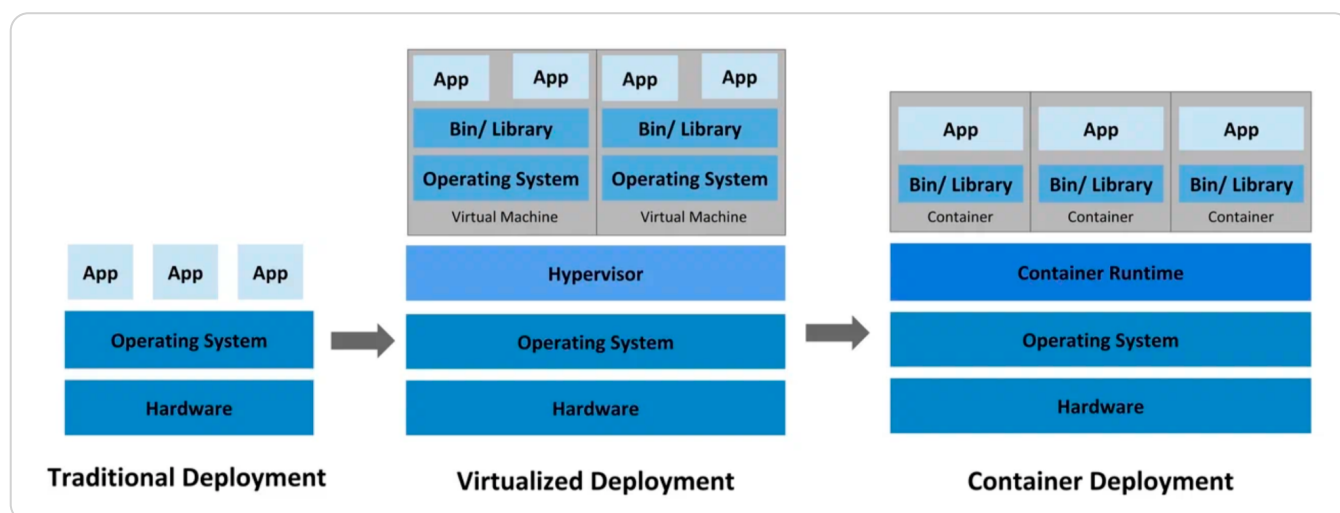


Рисунок 1. Сравнение традиционного развёртывания, виртуализации и контейнеризации с использованием Docker.

Источник: <https://shalaginov.com/>

ВИРТУАЛИЗАЦИЯ VS КОНТЕЙНЕРИЗАЦИЯ

Разберём плюсы и минусы виртуализации и контейнеризации.

	Виртуализация	Контейнеризация
Плюсы	• Лучшая изоляция между приложениями; • запуск разных ОС на одном сервере	• Легковесность: не нужно дублировать ОС; • быстрое развёртывание и масштабирование; • требует меньше ресурсов
Минусы	• Высокие расходы на ресурсы: для каждой VM нужна отдельная ОС; • более долгий запуск по сравнению с контейнерами	• Изоляция слабее, чем у виртуальных машин (используется общее ядро); • нужна совместимость с ОС хоста: нельзя запустить Windows-контейнеры на Linux без дополнительных решений

ПОЧЕМУ DOCKER СТАЛ СТАНДАРТОМ

Стать стандартом отрасли платформе Docker помогли эти преимущества.

- **Стандартизированное API** — единый интерфейс для работы с контейнерами, который упрощает автоматизацию и интеграцию в CI/CD-пайплайны. Команды `docker build`, `docker pull`, `docker run` одинаковы везде.
- **Полноценная экосистема** — Docker Hub и private registry для хранения и распространения образов. Вся сложная работа по управлению контейнерами берёт на себя Docker Daemon, работая в фоновом режиме.
- **Упрощённая разработка и развёртывание** — одинаковые среды на любой ОС. Благодаря layered filesystem — слоистой файловой системе — образы эффективно хранятся и загружаются.
- **Лёгкость в масштабировании** — интеграция с Kubernetes и другими оркестраторами, что сделало Docker незаменимым в облачных средах.
- **Мультиплатформенность** — работает на Windows, macOS и Linux.

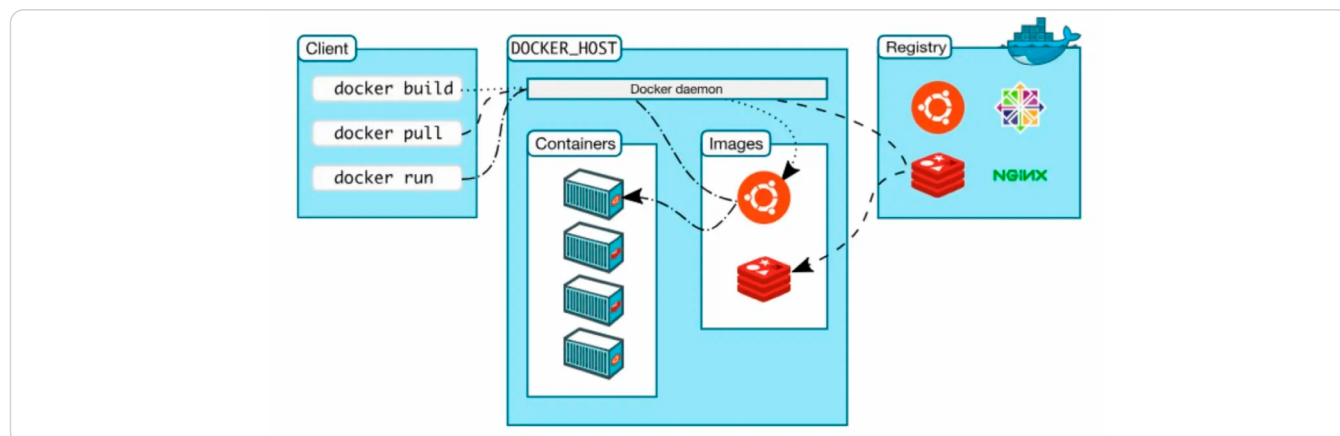


Рисунок 2. Архитектура Docker и взаимодействие её основных компонентов. Источник: <https://dishcuss.com/>

ОСНОВНЫЕ СУЩНОСТИ DOCKER

ОПРЕДЕЛЕНИЯ

Dockerfile — это **рецепт** (инструкция) для создания образа.

Описывает, какие зависимости и настройки нужны для контейнера.

Образ (Image) — это **шаблон** (слепок файловой системы), созданный из Dockerfile.

Содержит все зависимости: библиотеки, исполняемые файлы и конфигурации.

Контейнер (Container) — это **запущенный экземпляр** образа.

Работает изолированно, но использует ресурсы хоста.

Docker Daemon — фоновый процесс, управляющий контейнерами и образами.

Обрабатывает команды **docker build**, **docker run**, **docker pull**.

Docker Registry — репозиторий для хранения образов. Позволяет загружать (**docker pull**) и отправлять (**docker push**) образы.

Минимальный Dockerfile

```
FROM python:3.9
COPY app.py /app/app.py
CMD ["python", "/app/app.py"]
```

Образы хранятся в Docker Registry (например, Docker Hub) и запускаются как контейнеры с помощью docker run.

Контейнеры можно:

- запускать командой docker start,
- останавливать командой docker stop,
- удалять командой docker rm.

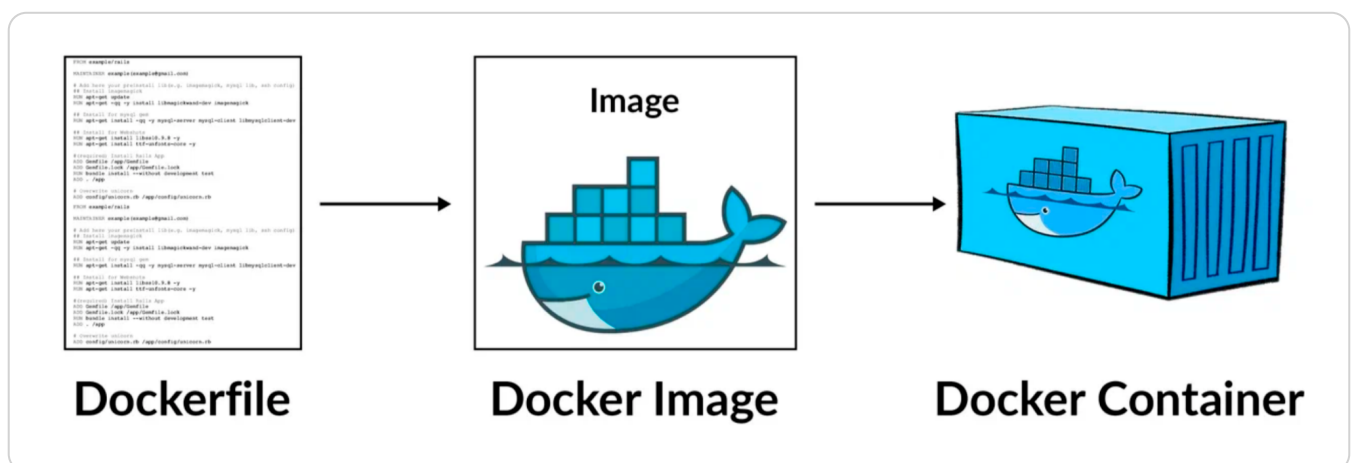


Рисунок 3. Жизненный цикл Docker-приложения: Dockerfile → образ → контейнер. Источник: <https://dishcuss.com/>

ЖИЗНЕННЫЙ ЦИКЛ DOCKER

Чтобы упаковать приложение в контейнер и начать им пользоваться, нужно пройти несколько шагов.

1. Пишем Dockerfile.
2. Собираем образ — `docker build`.
3. Загружаем образ в Registry — `docker push`, если нужно.
4. Запускаем контейнер — `docker run`.
5. Управляем контейнером — `start / stop / restart / rm`.

CHEAT SHEET: ЧАСТО ИСПОЛЬЗУЕМЫЕ КОМАНДЫ

```
# Список всех контейнеров (включая остановленные)
docker ps -a

# Построение Docker-образа из Dockerfile
docker build -t <image_name> .

# Запуск контейнера в фоновом режиме
docker run -d <image_name>

# Запуск контейнера с интерактивным терминалом
docker run -it <image_name> /bin/bash

# Список всех образов
docker images

# Остановка контейнера
docker stop <container_id>

# Удаление контейнера
docker rm <container_id>

# Удаление образа
docker rmi <image_id>

# Получение информации о контейнере
docker inspect <container_id>

# Запуск команд в работающем контейнере
docker exec -it <container_id> <command>

# Просмотр логов контейнера
docker logs <container_id>

# Остановка всех контейнеров
docker stop $(docker ps -q)

# Удаление всех остановленных контейнеров
docker container prune

# Удаление всех ненужных образов (не используемых контейнерами)
docker image prune
```

```

# Публикация портов контейнера на хост
# Запросы с хоста на порт host_port будут проброшены
# на порт container_port внутри контейнера
docker run -p <host_port>:<container_port> <image_name>

# Просмотр статистики использования ресурсов контейнерами
docker stats

# Переименовать контейнер
docker rename <old_name> <new_name>

# Создание и запуск контейнера с томом
# Файлы из host_directory будут доступны внутри контейнера
# в директории container_directory
docker run -v <host_directory>:<container_directory> <image_name>

# Получение информации о системе Docker
docker info

# Запуск контейнера с ограничением по ресурсам (CPU и память)
docker run --memory="500m" --cpus="1.0" <image_name>

# Запуск контейнера в сети хоста
# (не нужно пробрасывать порты — контейнер использует сеть хоста)
docker run --net=host <image_name>

```

ВЗАИМОДЕЙСТВИЕ КОНТЕЙНЕРОВ И ХОСТА

Рассмотрим взаимодействие контейнеров и хоста.

Хост ↔ контейнер

- Проброс портов (`-p 8080:80`) для доступа к сервисам контейнера с хоста;
- монтирование директорий (`-v ./data:/app/data`) для обмена файлами между хостом и контейнером.

Контейнер ↔ контейнер

- По умолчанию все контейнеры в дефолтной bridge-сети видят друг друга по IP;
- для общения по именам сервисов создаются пользовательские сети (`docker network create`).

Контейнер → хост

- Через gateway дефолтной сети (`172.17.0.1`);
- через `host.docker.internal` (Docker Desktop, новые версии Linux).

Инсайт



По умолчанию контейнер ничего не хранит. Если удалить контейнер, то удалятся и все данные внутри него. Поэтому всё, что должно пережить перезапуск (модели, данные, логи), должно лежать в томе или на смонтированной директории хоста.

DOCKER В РАЗРАБОТКЕ

Docker меняет подход к разработке и решает задачи на самых разных этапах жизни приложения.

1. **Среда разработки.** Достаточно один раз описать окружение (в Dockerfile/Compose), и оно одинаково работает у всех. Больше нет «у меня не запускается» из-за разных версий Python и библиотек.
2. **Тестирование и CI/CD.** Легко поднимать сервисы в GitHub Actions и GitLab CI. При этом тесты проходят в том же окружении, что и production.
3. **Production-деплой.** Тот же образ запускается и локально, и на сервере, и в Kubernetes. Выкатывание новой версии — это замена контейнера.
4. **Микросервисы и изоляция.** Каждый сервис работает независимо, в отдельном контейнере. Легко контролировать ресурсы (CPU, RAM), нет конфликтов зависимостей между сервисами.

УПАКОВКА СЕРВИСА В DOCKER

Рассмотрим контейнеризацию на том же тестовом примере [из репозитория](#).

```
FROM python:3.12-slim

# Bring in a static uv binary (no pip install needed, single-
purpose layer).
COPY --from=ghcr.io/astral-sh/uv:0.9.6 /uv /uvx /usr/local/bin/

WORKDIR /app

ENV UV_COMPILE_BYTECODE=1 \
    PATH="/app/.venv/bin:$PATH"

# Install dependencies first so this layer is cached unless
pyproject.toml
# or uv.lock change (independent of application code changes).
COPY pyproject.toml uv.lock ./

RUN uv sync --frozen --no-install-project --no-dev

# Now copy the application code and install the project itself.
COPY service ./service

RUN uv sync --frozen --no-dev

EXPOSE 8000

CMD ["uvicorn", "service.main:app", "--host", "0.0.0.0", "--
port", "8000"]
```

Разберём работу кода по шагам.

1. Для создания контейнера используется официальный образ Python с нужной версией.
2. Устанавливается необходимая версия uv, и с её помощью настраивается виртуальное окружение с зависимостями проекта.
3. Копируются исходный код и файлы конфигураций приложения. На последнем шаге запускается сервис.

Создание образа

```
docker build -t week-1-demo-app .
```

Запуск контейнера с пробросом порта 8000

```
docker run -d -p 8000:8000 week-1-demo-app
```

После этого можно делать запросы к сервису тем же **curl**, что и раньше.

Почему **COPY pyproject.toml uv.lock ./**
и **RUN uv sync --frozen --no-install-project --no-dev**
идут до **COPY service ./service**



Каждая инструкция Dockerfile — это слой, и Docker кеширует слои. Зависимости меняются редко, а код — постоянно. При таком порядке правка одной строчки кода пересобирает только последние слои, а не переустанавливает все зависимости заново. Если же переставить строки местами, то каждая сборка будет занимать уже минуты, а не секунды.

Docker Compose

Один контейнер — это хорошо. Но реальный сервис — это как минимум приложение и база данных. А чаще ещё кеш, очередь и мониторинг. Поднимать это руками через **docker run** с десятком флагов в правильном порядке каждый раз — занятие не из лёгких.

ОПРЕДЕЛЕНИЕ

Docker Compose — инструмент для описания и запуска многоконтейнерных приложений. Он помогает описать несколько взаимосвязанных контейнеров, их взаимодействие, зависимости, сети и тома в одном YAML-файле.

Основная идея Docker Compose — собрать весь стек (приложение + базу + кеш и др.) в одном файле, чтобы управлять им как единым целым.

СТРУКТУРА DOCKER-COMPOSE .YAML

Три ключевые секции:

- **services** — контейнеры приложения;
- **volumes** — постоянное (персистентное) хранение данных;
- **networks** — изоляция и взаимодействие контейнеров.

```
version: '3.9'

services:
  app:
    image: myapp:latest
    build: ./app
    ports:
      - "8080:80"
    networks:
      - backend
    depends_on:
      - db

  db:
    image: postgres:15
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: password
      POSTGRES_DB: mydb
    volumes:
      - db_data:/var/lib/postgresql/data
    networks:
      - backend

volumes:
  db_data:

networks:
  backend:
```



Обрати внимание: сервис **app** обращается к базе просто по имени **db** — Compose поднимает свою сеть и настраивает DNS, так что имена сервисов работают как хостнеймы. При этом вписывать IP-адреса вручную не нужно.

ОСНОВНЫЕ КОМАНДЫ

```
# Запустить все сервисы в фоновом режиме (detached mode)
docker compose up -d

# Запустить все сервисы в активном режиме (отображает логи)
docker compose up

# Остановить и удалить все запущенные сервисы, сети и тома
docker compose down

# Остановить сервисы без удаления контейнеров
docker compose stop

# Запустить ранее остановленные сервисы
docker compose start

# Перезапустить все сервисы
docker compose restart

# Показать список запущенных сервисов
docker compose ps

# Показать логи всех сервисов
docker compose logs

# Показать логи и следить за обновлениями в реальном времени
docker compose logs -f

# Собрать образы для всех сервисов
docker compose build

# Собрать образы и запустить сервисы
docker compose up --build

# Скачать последние версии образов, указанных в compose-файле
docker compose pull

# Выполнить команду внутри запущенного контейнера
docker compose exec <service> <command>

# Пример: открыть shell внутри контейнера "app"
docker compose exec app bash
```



up с флагом **-d** запускает сервисы в фоне, а **logs -f** помогает «подписаться» на поток логов в реальном времени. Связка **docker compose up -d && docker compose logs -f** — самая частая команда при локальной разработке.

КОНФИГУРАЦИЯ НАШЕГО СЕРВИСА

В репозитории курса в файле `docker-compose.yaml` лежит конфигурация для поднятия сервиса. Рассмотрим её.

ОСНОВНЫЕ КОМАНДЫ

```
services:
  db:
    image: postgres:16-alpine
    container_name: week1-demo-db
    restart: unless-stopped
    env_file:
      - .env
    environment:
      POSTGRES_USER: ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
      POSTGRES_DB: ${DB_NAME}
    ports:
      - "${DB_PORT:-5432}:5432"
    volumes:
      - db-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${DB_USER} -d ${DB_NAME}"]
      interval: 5s
      timeout: 5s
      retries: 5
  app:
    build: .
    container_name: week1-demo-app
    restart: unless-stopped
    env_file:
      - .env
    environment:
      # Overrides the .env value on purpose: inside the compose network the
      # database is reachable at the "db" service name, not at the host used
      # for the local (no-docker) part of the lesson.
      DB_HOST: db
    ports:
      - "443:8000"
    depends_on:
      db:
        condition: service_healthy
volumes:
  db-data:
```

После запуска командой `docker compose up` сервис будет доступен на порту 443 именно потому, что в конфигурации указан проброс портов "`443:8000`": внешний порт 443 хоста ведёт на порт 8000 внутри контейнера, где слушает `uvicorn`.

Если кратко

- Терминал и SSH помогают перевести работу из среды «у меня на ноутбуке» туда, где есть ресурсы и где окружение похоже на production.
- Git решает проблему «у меня была другая версия» так, что история одна, общая и к любой её точке можно вернуться.
- Структура проекта, `venv` и `uv` с `lock`-файлом решают проблему «у меня другие библиотеки»: окружение перестаёт быть личной тайной каждого разработчика и становится частью репозитория.
- Docker убирает последнее «у меня другая система»: приложение едет вместе со всем своим окружением, а образ, собранный у тебя, ведёт себя одинаково на сервере, в CI и Kubernetes.
- Docker Compose превращает набор контейнеров в один стек, который поднимается одной командой.

Ни один из этих инструментов сам по себе не сделает модель точнее. Но именно они определяют, доедет ли твоя модель до пользователя. И сможет ли кто-нибудь, включая тебя через полгода, воспроизвести полученный результат.